

Имплементација погона за дводимензионалне видео игре користећи ЕКС архитектуру и програмски језик *Rust*

Никола Огњеновић

Факултет техничких наука у Новом Саду

10.09.2026.

Првобитна идеја и план

- Првобитна идеја: „Израда погона за видео игре користећи *Rust* и зашто је то лоша идеја”
- План: направити комплетан погон, укључујући и графички слој ниског нивоа, потпуно у *Rust*-у

Погони за видео игре

- Софтверски систем који обезбеђује заједничку инфраструктуру за развој игара (рендеровање, ресурси, улаз, звук, физика...)
- Слој апстракције између оперативног система и логики конкретне игре
- Историјски: логика игре и инфраструктура биле су тесно повезане
- Смањују време развоја и омогућавају фокус на дизајн игре, а не на решавање истих техничких проблема изнова

ЕКС архитектура и зашто се користи

- *Ентитети* - само идентификатори; *компоненте* - чисти подаци; *системи* - логика обраде
- Композиција уместо наслеђивања: нови тип објекта = нова комбинација компоненти уместо нове класе
- Предности: мања спрегнутост делова система, лакше проширење, боља искоришћеност кеша, лакше управљање великим бројем ентитета
- Природно се уклапа у *Rust*-ов модел власништва и позајмљивања

Хипотеза

- Могуће је развити функционалан ЕКС погон за 2Д видео игре искључиво у безбедном *Rust* коду
- Циљ: ЕКС језгро (ентитети, компоненте, системи), рендеринг слој и управљање ресурсима
- Небезбедни код је дозвољен само унутар спољних библиотека нижег нивоа
- Фокус рада: провера техничке изводљивости и анализа последица по архитектуру, једноставност и перформансе

Зашто *Rust*?

- Меморијска безбедност кроз власништво, позајмљивање и животне векове + грешке се хватају у компајлирању
- Избегавају се класични проблеми *C/C++* приступа (dangling pointers, double free, data race)
- *Rust* дозвољава и небезбедни код, али рад намерно испитује колико далеко се може стићи без њега у апликативном слоју

Ентитети: `id + generation`

- Ентитет = пар (`id`, `generation`)
- Идентификатори се рециклирају (`free_ids`)
- Генерација решава проблем вишећих референци на обрисане ентитете
- Компромис у односу на један `u64` или `UUID`: боља читљивост, кеш локалност и мањи меморијски трошак

Компоненте: складишта и *type erasure*

- Свака компонента има засебно складиште (`HashMapComponentStorage<T>`)
- *Type erasure* преко `dyn ComponentStorage` + `downcast` омогућава хетерогену колекцију складишта
- Јавни АПИ остаје статички типски безбедан, унутрашњост користи динамички диспеч
- `HashMap` приступ изабран због једноставности и лакше серијализације, а алтернативе (архетипови, *sparse set*, *SoA*) су правац за унапређење

Догађаји и системи

- `event.rs`: редови по типу догађаја (`EventQueue<E>`, FIFO)
- `system.rs`: системи имплементирају `run(&mut self, world)`
- `SystemExecutor` извршава системе секвенцијално, у детерминистичком редоследу
- Догађаји раздвајају намеру акције од саме промене стања

World: интеграциони слој и пример игре

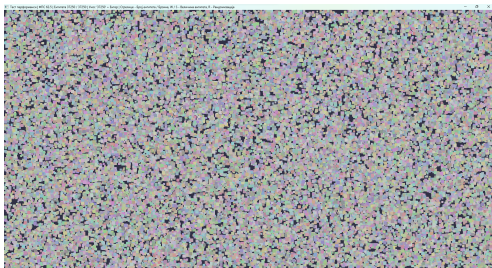
- World обједињује EntityManager, ComponentManager, EventManager
- Подржана серијализација/десеријализација стања у JSON формату
- Демонстрација: текстуална потезна борбена игра где су играч и непријатељи ентитети, без класних хијерархија

Рендеринг слој

- Изабран *wgpu*: мултиплатформски, небезбедни код остаје енкапсулиран унутар библиотеке
- Не намеће сопствени *game loop* нити фиксну организацију сцене и лако се уклапа у ЕКС систем
- Повезан са ЕКС-ом преко компонената `Transform2D` и `SpriteComponent`

Евалуациона апликација и резултати

- Стрес тест: велики број ентитета (квадрата) који се насумично крећу по прозору, мерење ФПС-а
- 37.000 ентитета у стабилних 60 ФПС
- 500.000 ентитета у ~ 2 ФПС, уз само $\sim 25\%$ GPU и $\sim 10\%$ CPU искоришћења
- Закључак: једнонитни систем добро подноси до пар десетина хиљада ентитета, простор за паралелизацију



Закључак

- Хипотеза потврђена: могуће је направити безбедан ЕКС и рендеринг слој у *Rust*-у уз *wgpu*
- Резултат: модуларан, читљив и меморијски безбедан погон за 2Д игре
- Правци даљег развоја: паралелно извршавање система, софистицираније складиштење компоненти, скриптовање (*Rhai/Lua*), физика (*Rapier*)
- Захвалница и питања